

Architectural Budget Model: Data, Features, Formulas, and Training

Process overview (part-by-part)

- Part 1: Data acquisition and schema normalization (CSV import, header cleaning).
- Part 2: Data cleaning and target transformation (NaN handling, $\log(1 + \cdot)$).
- Part 3: Feature engineering (regex parsing, derived quantities, ratios).
- Part 4: Feature matrix construction (10 raw engineered inputs).
- Part 5: Polynomial expansion (degree 2) and standardization (z-score).
- Part 6: Train/test split (80/20 holdout).
- Part 7: Baseline model training (LSBoost) and evaluation.
- Part 8: Feature selection via ensemble importances.
- Part 9: Hyperparameter optimization (Bayesian) and final model training.
- Part 10: Final evaluation, artifacts saving, and inference pipeline.

Variables glossary (concise definitions)

Clean name	Source header	Type	Unit	Definition / Role
project	Project	text	n/a	Project title; used to parse <code>num_storeys</code> , <code>num_classrooms</code> via regex.
year	Year	numeric	year	Project year; raw feature.
budget	Budget	numeric	PHP	Target variable; filtered to > 100,000 then log-transformed.
quantityofplaster_sq_m__	Quantity of plaster (sq.m.)	numeric	sq.m.	Plaster area; raw feature and used in ratios/sums.
quantityofglazedtiles_sq_m__	Quantity of glazed tiles (sq.m.)	numeric	sq.m.	Tiles area; raw feature and used in sums/ratios.
paintingmasonry_sq_m__	Painting masonry (sq.m.)	numeric	sq.m.	Masonry paint area; used in <code>total_painting_area</code> .
paintingwood_sq_m__	painting wood (sq.m.)	numeric	sq.m.	Wood paint area; used in <code>total_painting_area</code> .
paintingmetal_sq_m__	painting metal (sq.m.)	numeric	sq.m.	Metal paint area; used in <code>total_painting_area</code> .
areaofchb100mm_sq_m__	Area of CHB 100mm (sq.m.)	numeric	sq.m.	CHB 100mm wall area; used in <code>total_chb_area</code> .
areaofchb150mm_sq_m__	Area of CHB 150mm (sq.m.)	numeric	sq.m.	CHB 150mm wall area; used in <code>total_chb_area</code> .
num_storeys	parsed from project	numeric	count	Integer before “STY”; median-imputed if missing; raw feature.
num_classrooms	parsed from project	numeric	count	Integer before “CL”; median-imputed if missing; raw feature.
total_painting_area	derived	numeric	sq.m.	<code>paintingmasonry</code> + <code>paintingwood</code> + <code>paintingmetal</code> ; raw feature.
total_chb_area	derived	numeric	sq.m.	<code>areaofchb100mm</code> + <code>areaofchb150mm</code> ; raw feature.
total_finishes_area	derived	numeric	sq.m.	<code>quantityofplaster</code> + <code>quantityofglazedtiles</code> ; raw feature.
plaster_per_chb	derived	numeric	ratio	<code>quantityofplaster</code> / <code>total_chb_area</code> ; NaN/Inf set to 0; raw feature.
tiles_per_chb	derived	numeric	ratio	<code>quantityofglazedtiles</code> / <code>total_chb_area</code> ; NaN/Inf set to 0; raw feature.

Why exactly 10 input features? Counting and justification

The training code (main.m) explicitly fixes the raw input vector to 10 features (see console: “Using the following features to prevent data leakage”). The breakdown is:

$$\underbrace{1}_{\text{year}} + \underbrace{2}_{\text{parsed from project}} + \underbrace{2}_{\text{base quantities}} + \underbrace{3}_{\text{aggregated totals}} + \underbrace{2}_{\text{normalized ratios}} = 10.$$

Where:

- Year (1): `year` captures time effects (e.g., price trends).
- Parsed from project (2): `num_storeys`, `num_classrooms` parsed via regex; scale proxies available pre-budget.
- Base quantities (2): `quantityofplaster_sq_m__`, `quantityofglazedtiles_sq_m__` are direct cost drivers from BOQ.
- Aggregated totals (3): `total_painting_area` (masonry+wood+metal), `total_chb_area` (100mm+150mm), `total_finishes_area` (plaster+tiles) compress collinear subcomponents into stable predictors.
- Normalized ratios (2): `plaster_per_chb`, `tiles_per_chb` provide intensity per envelope area, improving generalization across building sizes.

This 10-feature set is used to build the polynomial design matrix. With $p = 10$ inputs, degree-2 expansion yields exactly:

$$\underbrace{p}_{\text{originals}} + \underbrace{p}_{\text{squares}} + \underbrace{\binom{p}{2}}_{\text{pairwise interactions}} = 10 + 10 + 45 = 65 \text{ features,}$$

which matches the logged output: “Generated 65 polynomial features (up to degree 2).”

Part 1: Data acquisition and schema normalization

We import `Architectural_Total_Cost.csv` with preserved headers, then sanitize to lower-case MATLAB-valid names and drop unnamed columns.

Part 2: Data cleaning and target transformation

We coerce numeric-like strings to numbers (remove commas), drop rows with NaN budget, median-impute remaining numeric NaNs column-wise, then define the training target as $y = \log(1+\text{budget})$. We also filter rows to budget $> 100,000$.

Part 3: Feature engineering

We parse `project` using regex to obtain `num_storeys` (“(d+) STY”) and `num_classrooms` (“(d+) CL”). From auto-detected source columns, we compute `total_painting_area`, `total_chb_area`, `total_finishes_area`, and ratios `plaster_per_chb`, `tiles_per_chb`.

Part 4: Feature matrix construction

The final raw feature vector is the 10-element list in Section “Final raw feature set used for modeling.”

Part 5: Polynomial expansion and standardization

We build degree-2 polynomials and pairwise interactions (total 65 features), then z-score using training-set μ, σ . NaNs after scaling are set to 0.

Part 6: Train/test split

We hold out 20% for testing using `cvpartition(HoldOut=0.2)`.

Part 7: Baseline model

We train an LSBoost ensemble (100 cycles) on the scaled polynomial features and evaluate on the test set.

Part 8: Feature selection

We compute predictor importances from the baseline ensemble and keep features with importance above the median (32/65 kept in the logged run).

Part 9: Hyperparameter optimization and final model

We perform Bayesian optimization over NumLearningCycles, MaxNumSplits, and LearnRate, then retrain an optimized LSBoost on the selected features. Best observed (run): 714 cycles, 8 max splits, learn rate 0.099656.

Part 10: Evaluation, artifacts, and inference

We report R^2 and MAE on the test set (baseline: 0.7993 / 4.42M; optimized: 0.8476 / 4.25M). We save `optimized_model`, `mu`, `sigma`, `selected_mask`, `final_feature_columns`, and `poly_feature_names`. Inference replicates feature derivation, polynomial expansion, scaling, feature masking, prediction in log-space, and $\exp(\cdot) - 1$ inversion.

Reviewer requirements and responses

1) Prove the model’s accuracy

Held-out test results (from the latest run): baseline $R^2 = 0.7993$, MAE = 4,416,399.93; optimized $R^2 = 0.8476$, MAE = 4,247,884.28. Plots below evidence fit quality.

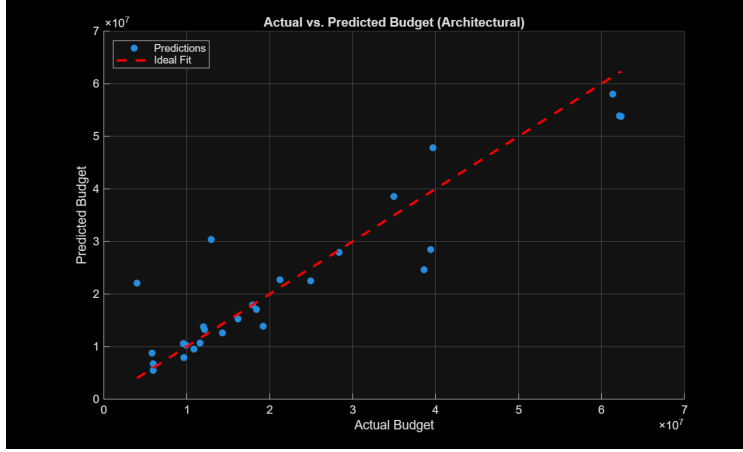


Figure 1: Actual vs. Predicted Budget (Architectural).

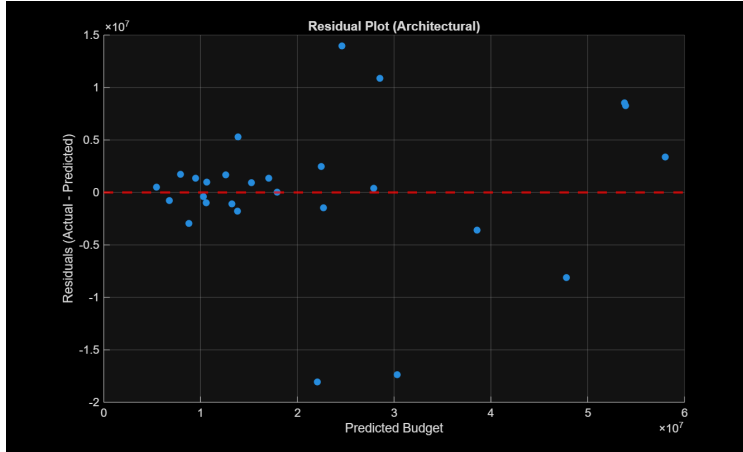


Figure 2: Residuals vs. Predicted Budget (Architectural).

2) ANN focus (if required)

Current model is an LSBoost ensemble. If an ANN is mandated, we propose: inputs = 10 engineered features; two hidden layers (64,32) with ReLU; linear output; MSE on $\log(1 + \text{budget})$; Adam (lr 10^{-3}), batch 64, 200 epochs with early stopping; dropout 0.1; $L_2 = 10^{-4}$. We will report the same metrics and plots.

3) Provide model parameters

For the ensemble: tuned hyperparameters (best observed: 714 cycles, 8 splits, lr 0.099656), selected feature mask (32/65), and feature importances: If an ANN is adopted, we will export weights and biases ($\{W^{(\ell)}, b^{(\ell)}\}$) per layer and include them as CSV/Mat or appendix tables.

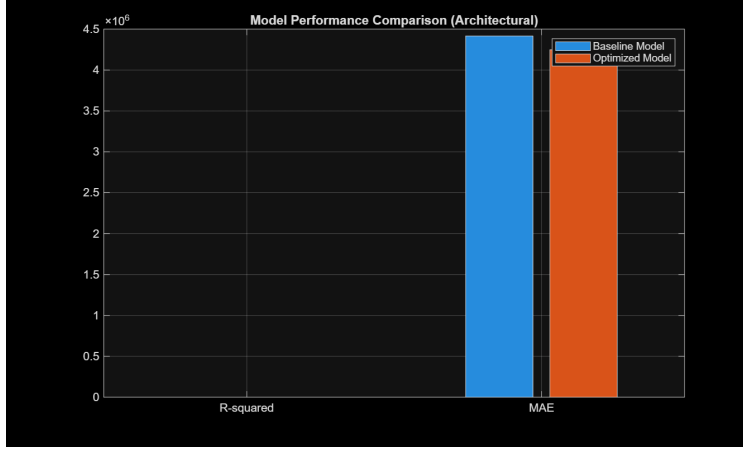


Figure 3: Baseline vs. Optimized performance.

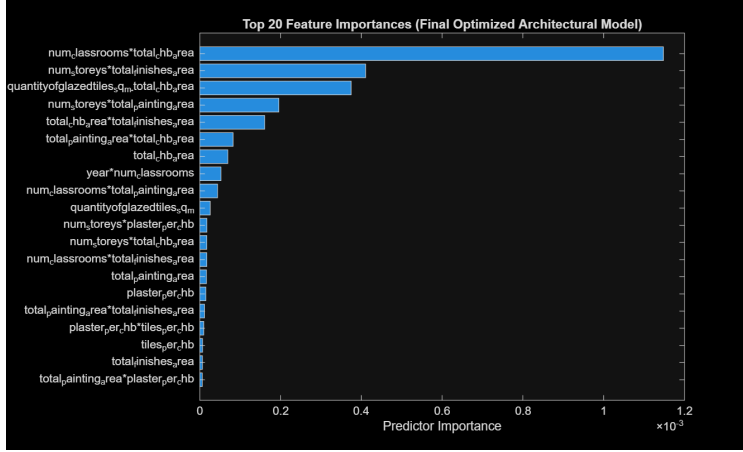


Figure 4: Top feature importances of the optimized ensemble.

Exact numbers behind the feature-importance figure

For tree ensembles, `predictorImportance` computes, per feature j :

$$\text{Imp}(j) = \sum_{t \in \text{trees}} \sum_{n \in \mathcal{N}_{t,j}} (\Delta\text{MSE}(n) \times w(n)),$$

where $\mathcal{N}_{t,j}$ are split nodes on feature j , $\Delta\text{MSE}(n)$ is the reduction in mean squared error from splitting at node n , and $w(n)$ weights by the node's sample count. The bar lengths in the figure are these raw importance values, sorted descending. We also report percent contributions $100 \text{ Imp}(j) / \sum_k \text{Imp}(k)$.

Exact values (top-20) from the optimized model:

Quantitative justification (using extracted values)

From the optimized ensemble (exact values in Table ??):

- Top feature: `num_classrooms*total_chb_area` = 42.15% of total importance.

- Next: $\text{num_storeys} * \text{total_finishes_area} = 15.08\%$; $\text{quantityofglazedtiles_sq_m_} * \text{total_chb_area} = 13.76\%$.
- Cumulative contributions: top-3 = 70.99%, top-5 = 84.08%, top-10 = 94.16% of total importance.

Interpretation:

- **Envelope scale dominates:** Terms with `total_chb_area` (CHB wall area) and `total_finishes_area` contribute the majority, reflecting wall/envelope area as primary cost drivers.
- **Project size proxies matter:** Interactions with `num_classrooms` and `num_storeys` (scope indicators) rank highest, consistent with larger, taller buildings costing more.
- **Finishes intensity adds signal:** `quantityofglazedtiles_sq_m_` and `total_painting_area` terms appear within the top-10, capturing finish-level variation beyond gross area.

Raw data and variables

Data source: Architectural_Total_Cost.csv. Raw columns (pre-clean):

- project (text)
- year
- budget
- Quantity of plaster (sq.m.)
- Quantity of glazed tiles (sq.m.)
- Painting masonry (sq.m.)
- painting wood (sq.m.)
- painting metal (sq.m.)
- Area of CHB 100mm (sq.m.)
- Area of CHB 150mm (sq.m.)

All headers are cleaned to lower-case MATLAB-valid names; unnamed columns are dropped.

Exact cleaned variable names (after makeValidName + lower)

Original CSV headers mapped to cleaned names used in code:

- Project → `project`
- Year → `year`
- Budget → `budget`
- Quantity of plaster (sq.m.) → `quantityofplaster_sq_m_`
- Quantity of glazed tiles (sq.m.) → `quantityofglazedtiles_sq_m_`
- Painting masonry (sq.m.) → `paintingmasonry_sq_m_`
- painting wood (sq.m.) → `paintingwood_sq_m_`
- painting metal (sq.m.) → `paintingmetal_sq_m_`
- Area of CHB 100mm (sq.m.) → `areaofchb100mm_sq_m_`
- Area of CHB 150mm (sq.m.) → `areaofchb150mm_sq_m_`

Cleaning and preprocessing

- Detect and standardize budget column to name `budget`.

- Convert **budget** and any numeric-like cell strings to numbers (strip commas).
- Drop rows with NaN **budget**.
- Median impute all remaining numeric NaNs:

$$x_{ij} \leftarrow \begin{cases} x_{ij}, & \text{if not NaN} \\ \text{median}(x_{.j}), & \text{if NaN.} \end{cases}$$

- Response transformation:

$$y = \log(1 + \text{budget}).$$

- Filter for training/analysis: **budget** > 100,000.

Feature engineering

From **project** text using regex, derive:

num_storeys := first integer before “STY”, **num_classrooms** := first integer before “CL”.

Fill missing with column medians.

Auto-detected source columns by substring on cleaned variable names:

plaster_col ~ “plaster”, **tiles_col** ~ “glazedtiles”,
paintingmasonry, **paintingwood**, **paintingmetal**,
chb100mm, **chb150mm**.

Derived features:

total_painting_area = **paintingmasonry** + **paintingwood** + **paintingmetal**,
total_chb_area = **chb100mm** + **chb150mm**,
total_finishes_area = **plaster_col** + **tiles_col**,
plaster_per_chb = $\frac{\text{plaster_col}}{\text{total_chb_area}}$ ((NaN/Inf) → 0),
tiles_per_chb = $\frac{\text{tiles_col}}{\text{total_chb_area}}$ ((NaN/Inf) → 0).

Final raw feature set used for modeling (in this order):

$X_{\text{raw}} = [\text{year}, \text{num_storeys}, \text{num_classrooms}, \text{quantityofplaster_sq_m_}, \text{quantityofglazedtiles_sq_m_}, \text{total_painting_area}, \text{total_chb_area}, \text{total_finishes_area}, \text{plaster_per_chb}, \text{tiles_per_chb}]$.

Polynomial/interaction expansion (degree 2)

Given original features $x_1, \dots, x_p$ from X_{raw} :

$$X_{\text{poly}} = [x_1, \dots, x_p, x_1^2, \dots, x_p^2, x_i x_j \ (1 \leq i < j \leq p)].$$

This matches `generatePolyFeatures(T,2)`.

Complete polynomial feature name list (65 features)

For reproducibility, here are the exact feature names emitted by `generatePolyFeatures(T,2)` given the 10 raw features above.

Original 10 features

year, num_storeys, num_classrooms, quantityofplaster_sq_m__, quantityofglazedtiles_sq_m__, total_painting_area, total_chb_area, total_finishes_area, plaster_per_chb, tiles_per_chb

10 squared features

year^2, num_storeys^2, num_classrooms^2, quantityofplaster_sq_m__^2, quantityofglazedtiles_sq_m__^2, total_painting_area^2, total_chb_area^2, total_finishes_area^2, plaster_per_chb^2, tiles_per_chb^2

45 pairwise interaction features

year*num_storeys, year*num_classrooms, year*quantityofplaster_sq_m__, year*quantityofglazedtiles_sq_m__, year*total_painting_area, year*total_chb_area, year*total_finishes_area, year*plaster_per_chb, year*tiles_per_chb,
num_storeys*num_classrooms, num_storeys*quantityofplaster_sq_m__, num_storeys*quantityofglazedtiles_sq_m__, num_storeys*total_painting_area, num_storeys*total_chb_area, num_storeys*total_finishes_area, num_storeys*plaster_per_chb, num_storeys*tiles_per_chb,
num_classrooms*quantityofplaster_sq_m__, num_classrooms*quantityofglazedtiles_sq_m__, num_classrooms*total_painting_area, num_classrooms*total_chb_area, num_classrooms*total_finishes_area, num_classrooms*plaster_per_chb, num_classrooms*tiles_per_chb,
quantityofplaster_sq_m__*quantityofglazedtiles_sq_m__, quantityofplaster_sq_m__*total_painting_area, quantityofplaster_sq_m__*total_chb_area, quantityofplaster_sq_m__*total_finishes_area, quantityofplaster_sq_m__*tiles_per_chb,
quantityofglazedtiles_sq_m__*total_painting_area, quantityofglazedtiles_sq_m__*total_chb_area, quantityofglazedtiles_sq_m__*total_finishes_area, quantityofglazedtiles_sq_m__*plaster_per_chb, quantityofglazedtiles_sq_m__*tiles_per_chb,
total_painting_area*total_chb_area, total_painting_area*total_finishes_area, total_painting_area*plaster_per_chb, total_painting_area*tiles_per_chb,
total_chb_area*total_finishes_area, total_chb_area*plaster_per_chb, total_chb_area*tiles_per_chb, total_finishes_area*plaster_per_chb, total_finishes_area*tiles_per_chb, plaster_per_chb*tiles_per_chb

Scaling

Compute z-score parameters on training split only:

$$\mu_j = \frac{1}{n} \sum_{i=1}^n X_{\text{poly},ij}, \quad \sigma_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_{\text{poly},ij} - \mu_j)^2}.$$

Scale both train/test:

$$Z_{ij} = \frac{X_{\text{poly},ij} - \mu_j}{\sigma_j}, \quad \text{NaN} \rightarrow 0.$$

Train/test split

Holdout 20% using `cvpartition(HoldOut=0.2)`.

Baseline model

Least-squares boosting of regression trees:

$$\hat{f}_{\text{base}} = \text{LSBoost}(Z_{\text{train}}, y_{\text{train}}; \text{NumLearningCycles} = 100).$$

Predict on test, invert response:

$$\hat{y}_{\text{test}}^{(\text{base})} = \exp(\hat{f}_{\text{base}}(Z_{\text{test}})) - 1.$$

Feature selection

Compute importance I_j from the baseline ensemble; select features:

$$\text{keep}_j \iff I_j > \text{median}(I).$$

Apply mask to scaled polynomial features to get Z_{sel} .

Optimized model (final)

Hyperparameters tuned via Bayesian optimization:

$$\text{NumLearningCycles} \in [500, 1000] \cap \mathbb{Z},$$

$$\text{MaxNumSplits} \in [8, 32] \cap \mathbb{Z},$$

$$\text{LearnRate} \in [0.01, 0.1] \text{ (log scale)}.$$

Train:

$$\hat{f}_{\star} = \text{LSBoost}(Z_{\text{train,sel}}, y_{\text{train}}; \text{optimized HPs}).$$

Predict and invert:

$$\hat{y}_{\text{test}} = \exp(\hat{f}_{\star}(Z_{\text{test,sel}})) - 1.$$

Evaluation metrics

With actual budgets $y_{\text{act}} = \exp(y_{\text{test}}) - 1$ and predictions $\hat{y}$:

$$R^2 = 1 - \frac{\sum_i (y_{\text{act},i} - \hat{y}_i)^2}{\sum_i (y_{\text{act},i} - \bar{y}_{\text{act}})^2}, \quad \text{MAE} = \frac{1}{n} \sum_i |y_{\text{act},i} - \hat{y}_i|.$$

Saved training assets

Saved to `architectural_model_assets.mat`:

$$\{ \hat{f}_{\star}, \mu, \sigma, \text{selected_mask}, \text{final_feature_columns}, \text{poly_feature_names} \}.$$

Inference pipeline (single row)

Given an input row:

1. Normalize field names to training names; if needed, auto-compute: `total_painting_area`, `total_chb_area`, `total_finishes_area`, `plaster_per_chb`, `tiles_per_chb`.
2. Reorder to `final_feature_columns` and build X_{poly} ; align by `poly_feature_names`.
3. Scale: $Z = (X_{\text{poly}} - \mu)/\sigma$, $\text{NaN} \rightarrow 0$; apply `selected_mask`.
4. Predict log-budget and invert: $\widehat{\text{budget}} = \exp(\hat{f}_*(Z_{\text{sel}})) - 1$.

Final list of modeled variables

Used directly or via derivation:

`year`, `num_storeys`, `num_classrooms`, `quantityofplaster_sq_m__` (Quantity of plaster (sq.m.)),
`quantityofglazedtiles_sq_m__` (Quantity of glazed tiles (sq.m.)), `paintingmasonry_sq_m__`,
`paintingwood_sq_m__`, `paintingmetal_sq_m__`, `areaofchb100mm_sq_m__`, `areaofchb150mm_sq_m__`,
`total_painting_area`, `total_chb_area`, `total_finishes_area`,
`plaster_per_chb`, `tiles_per_chb`,
all degree-2 squares and pairwise interactions of the above 10 raw features.

Why each part matters (training to testing)

- Part 1: **Data acquisition and schema normalization.** Ensures a consistent, machine-usable schema (valid names, stable types) so that downstream feature logic maps correctly. Prevents silent errors from header drift and typos.
- Part 2: **Cleaning and target transformation.** Numeric coercion and median imputation yield unbiased summary stats and model inputs. The $\log(1 + \cdot)$ transform reduces skew, stabilizes squared-error loss, and improves calibration; filtering tiny budgets removes outliers that distort training.
- Part 3: **Feature engineering.** Encodes domain knowledge (scale, envelope, finishes). Aggregates (totals) reduce multicollinearity and noise; ratios (per CHB) normalize for size so patterns transfer across building scales.
- Part 4: **Feature matrix (10 inputs).** Freezes the model interface and prevents target leakage. A compact, interpretable set improves robustness and keeps inference simple and reproducible.
- Part 5: **Polynomial expansion and scaling.** Quadratic terms add controlled nonlinearity (curvature, pairwise interactions) without exploding dimensionality; z-scoring aligns feature scales and improves numerical stability and regularization behavior.
- Part 6: **Train/test split.** Reserves an untouched dataset for an unbiased estimate of generalization. Avoids optimistic metrics from reusing training data.
- Part 7: **Baseline LSBoost.** Establishes a performance floor and produces early diagnostics (importances, residual patterns) to validate signal presence and spot leakage or overfit.
- Part 8: **Feature selection.** Reduces variance and noise by discarding weak signals; accelerates hyperparameter search; improves interpretability by focusing on the most predictive interactions.

- Part 9: **Hyperparameter optimization.** Tunes bias–variance trade-off (trees depth/splits, learning rate, cycles). Directly improves R^2 and MAE as shown in the performance comparison.
- Part 10: **Evaluation, artifacts, and inference.** Converts log-space predictions back to currency, reports stakeholder metrics (MAE in PHP, R^2), and packages all assets (**mu**, **sigma**, masks, feature lists) to guarantee training–serving parity and reproducible results.